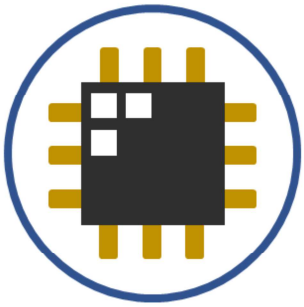
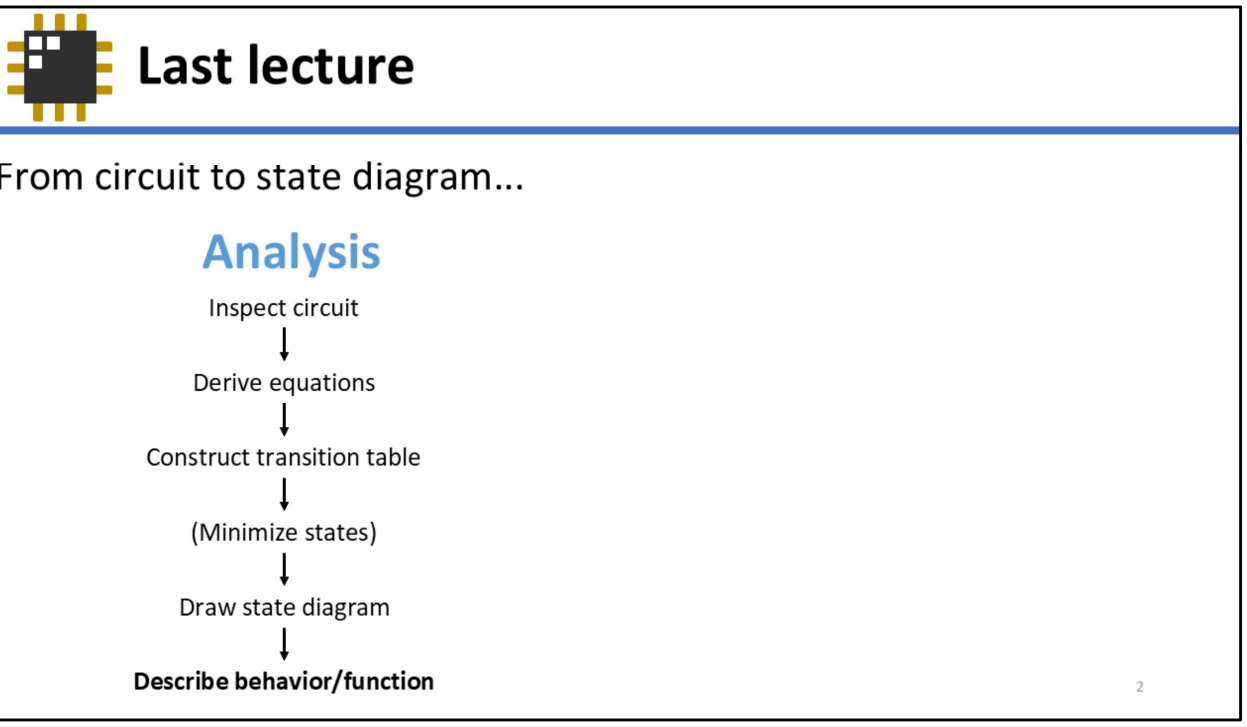


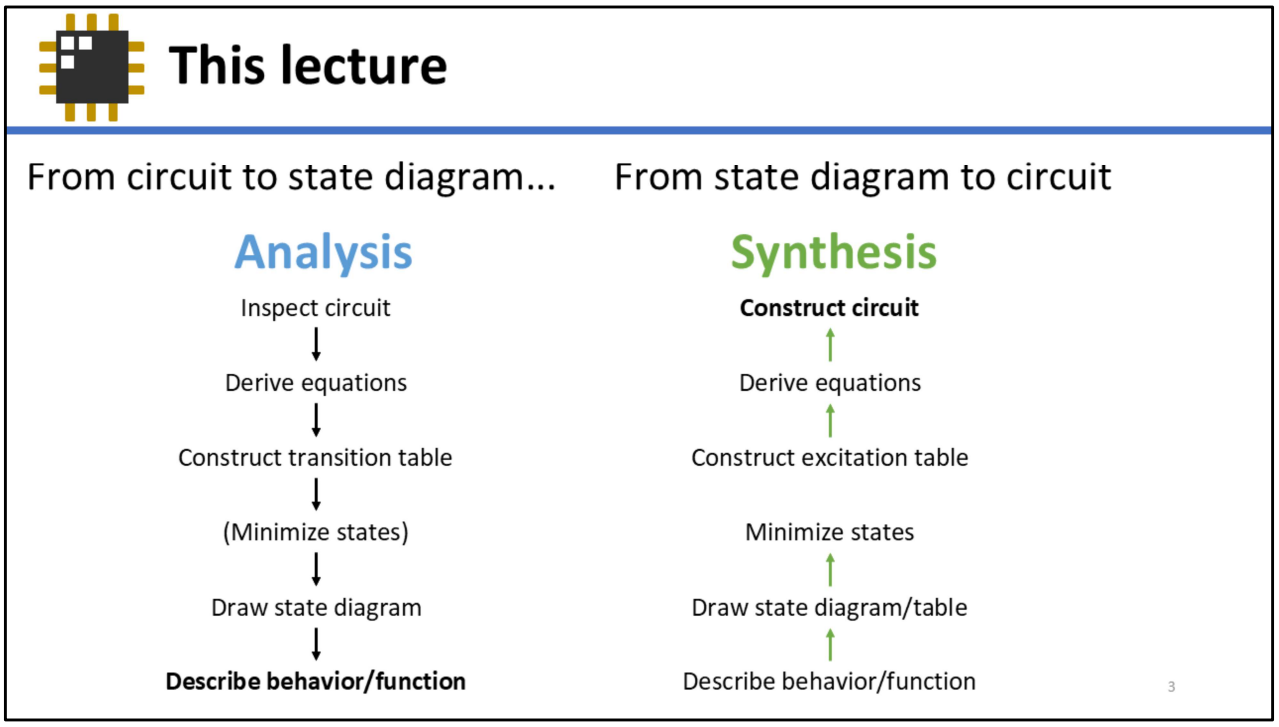
EEE 143: Switching Theory and Digital Logic Design

Lecture 11: Synthesis of Synchronous Sequential Circuits

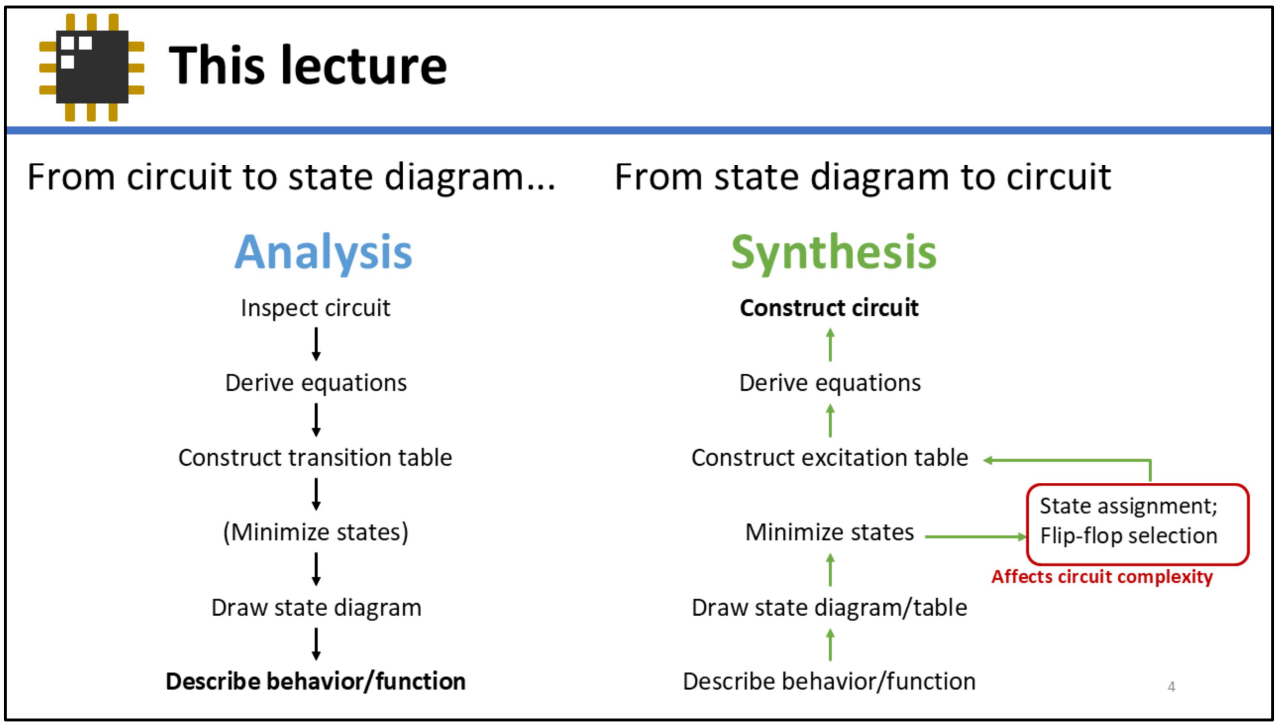
1



In our last lecture, we discussed the step-by-step process of analyzing sequential circuits. This begins with inspecting the circuit to identify its components, such as inputs, outputs, and flip-flops, and understanding how they are connected. From there, we derive the input and output equations using Boolean algebra based on the logic gates present in the circuit. Once we have these equations, we construct a transition table that shows all possible current states and inputs, along with their corresponding next states. To simplify the design, we minimize the number of states by identifying equivalent states. After this, we draw the state diagram to visually represent the transitions between states based on the inputs. Finally, we analyze and describe the output behavior of the circuit, determining how the outputs change depending on the states and inputs.



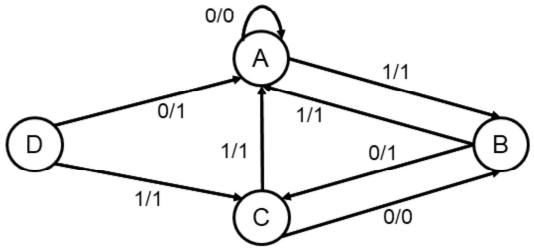
Now, for this lecture, we will focus on designing the circuit from scratch. The process begins with a defined behavior/function, which serves as the foundation for the design. From this, we construct the state table and state diagram to map out how the circuit should transition between states based on inputs. Once the table is established, we proceed to minimize the number of states to simplify the overall design. Next, we build the excitation table to determine the required flip-flop inputs for each transition. Using this information, we derive the Boolean equations for the flip-flop inputs and any outputs. Finally, we use all the derived data to construct the logic circuit, completing the design process.



After minimizing the states, we move on to the state assignment and flip-flop selection. In this step, we assign binary values to each state and choose the type of flip-flop to use. These choices affect the complexity and efficiency of the resulting circuit, as they determine the form of the excitation and output equations.

State Assignment

- In a state diagram or table, states may be denoted by symbols such as A, B, C … S0, S1, S2… RED, YELLOW, GREEN, …
- These must eventually be represented by 1s and 0s

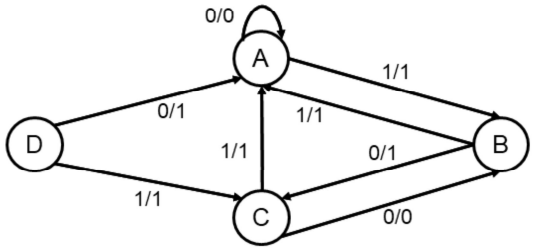


- How many flip-flops do you need to represent all states?
- How would you assign a binary sequence to each state?

In a state diagram or table, we may use various symbols such as letters, numbers, or descriptive names to represent each state. However, since digital circuits operate using binary data, each symbolic state must eventually be assigned a unique binary code made up of 1s and 0s before the state machine can be implemented in hardware.

State Assignment

- In a state diagram or table, states may be denoted by symbols such as A, B, C … S0, S1, S2… RED, YELLOW, GREEN, …
- These must eventually be represented by 1s and 0s



- How many flip-flops do you need to represent all states?
- How would you assign a binary sequence to each state?

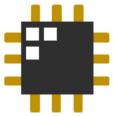
Recap

- A flip-flop is a memory element that stores *one* bit – either a 1 or a 0
- **In a state diagram, each state (circle) is a unique combination of flipflop outputs in the circuit**
- Flip-flops don't do anything until the clock ticks
  - Inputs are ignored except when they are sampled by the clock
  - Sampling takes place on the edge of the clock (here we assume rising/positive edge)
  - We move states (circles) only when the trigger event occurs

The number of flip-flops needed depends on how many states must be represented. Since each flip-flop stores one bit, a total of *n* flip-flops can represent up to 2^n unique states. Therefore, the minimum number of flip-flops required can be calculated using the formula:

$$\#\ of\ flip-flops = log_2(\#\ of\ states).$$

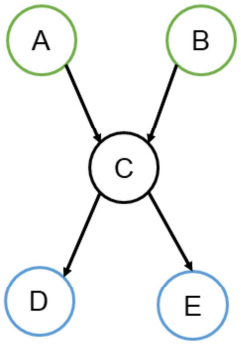
Once the number of flip-flops is determined, each state must be assigned a unique binary code, this process is called **state assignment**. There are basic guidelines for doing this, which will be discussed on the next slide.



State Assignment

Basic guidelines:

- Each state should have a unique binary assignment
- States having the same next state for a given input condition should have logically adjacent (1-bit change) assignments as much as possible
- The next states of a single state should have assignments that are logically adjacent as much as possible



Now, let's go over the basic guidelines for state assignment.

First, **each state must have a unique binary assignment** to ensure that the circuit can accurately identify and distinguish one state from another.

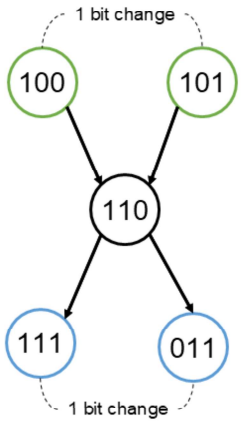
Second, **states that share the same next state for a given input** should be assigned binary codes that are **logically adjacent**—meaning their binary representations differ by only one bit. This helps simplify the logic, as transitions between these states require minimal changes in the circuit.

Lastly, **the next states of a single state** should also be **logically adjacent whenever possible**. This further reduces circuit complexity by minimizing the logic needed to handle transitions from one state to multiple others.

State Assignment

Basic guidelines:

- Each state should have a unique binary assignment
- States having the same next state for a given input condition should have logically adjacent (1-bit change) assignments as much as possible
- The next states of a single state should have assignments that are logically adjacent as much as possible



Following these guidelines can lead to a simpler overall circuit implementation of the state diagram:

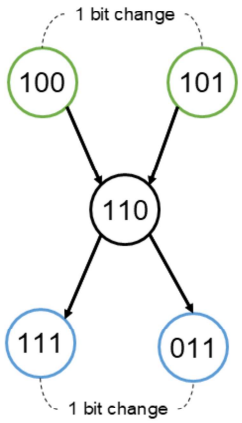
- 1-bit change ⇨ fewer flipflop transitions ⇨ simpler equations ⇨ simpler circuit!

Following the state assignment guidelines helps simplify the overall circuit design. When states are assigned using **1-bit changes** (logically adjacent), transitions between them require **fewer flip-flop changes**. This reduces the complexity of the logic needed to control those transitions, leading to **simpler Boolean equations** and ultimately a **simpler and more efficient circuit implementation**.

State Assignment

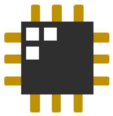
Basic guidelines:

- Each state should have a unique binary assignment
- States having the same next state for a given input condition should have logically adjacent (1-bit change) assignments as much as possible
- The next states of a single state should have assignments that are logically adjacent as much as possible



These are just *guidelines*, though, not a *strict* requirement.
Sometimes, the problem would already indicate how the states are assigned.

Following the state assignment guidelines helps simplify the overall circuit design. When states are assigned using **1-bit changes** (logically adjacent), transitions between them require **fewer flip-flop changes**. This reduces the complexity of the logic needed to control those transitions, leading to **simpler Boolean equations** and ultimately a **simpler and more efficient circuit implementation**. But do note that these are just guidelines and not a strict requirement.

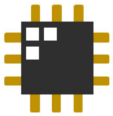


Flip-flop selection

- Depends on the application
- Depends on what you have available
- Typically given in a problem, unless otherwise stated

- You will *probably* stick to D or *possibly* JK flip-flops
  - JK is more flexible than SR
  - T can be used if J = K for each JK flip-flop

Flip-flop selection depends on the application requirements and the available components. In most problems, the type of flip-flop is either specified or assumed. You'll typically use **D flip-flops** for their simplicity, or sometimes **JK flip-flops** for more flexibility. **SR flip-flops** are rarely used because they're never simpler than JK. **T flip-flops** can be used if, in a JK flip-flop, **J = K** for all transitions. Choosing the right flip-flop can impact the efficiency of your circuit design.



Synthesis Procedure

1. Describe behavior/function
2. Draw the state diagram/table
3. Minimize states
4. Assign state variables
5. Choose flip-flops
6. Construct excitation table
7. Derive next-state and output functions
8. Construct the logic diagram/circuit
9. *Test the circuit (e.g. simulation)*

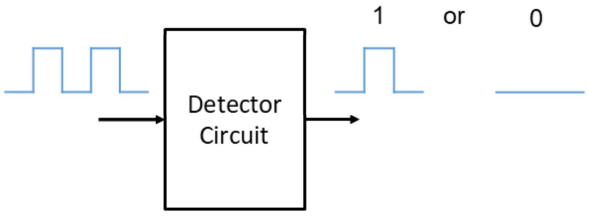
The **synthesis procedure** is a step-by-step method used to design a sequential circuit. First, you describe what the circuit should do—its purpose and how the outputs should respond to the inputs. Then, you draw a state diagram or state table to show how the system moves from one state to another based on different input conditions. After that, you try to reduce the number of states by combining any that behave the same, which makes the design simpler. Next, you assign binary codes (1s and 0s) to each state so the circuit can understand them.

Once the states are assigned, you choose the type of flip-flop to use—usually a D or JK flip-flop, depending on what's available or what the problem gives you. Then, you create an excitation table to figure out what input each flip-flop needs to make the correct transitions between states. From this, you write the equations for the next state and the output, using Boolean algebra or Karnaugh maps to make them simpler. After all that, you build the circuit using logic gates and flip-flops based on your equations. Lastly, you test the circuit, either in a simulator or with real hardware, to make sure it works as intended. This process helps you build a correct and efficient circuit.

Example 1

Beginning of message detector:

Design a sequential logic circuit that outputs a 1 (permanently) when three consecutive ones are present in a digital line. Use positive edge-triggered D flip-flops. Assume a Mealy machine behavior.



Let's look at our first example. We need to design a sequential logic circuit that **permanently outputs 1** once it detects **three consecutive 1s** on a digital input line. For this design, we'll use **positive edge-triggered D flip-flops**.

1.1 Describe behavior/function

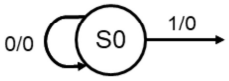
- Let X = input, Z = output
- Assume a starting or reset state, S0

Present State	Next State / Z	
	X = 0	X = 1
S0		

Let's begin by defining the system's behavior. Assume X represents the input and Z represents the output. We start with the reset state, labeled S0. In this state, we need to describe how the output Z behaves and determine the next state for each possible input value of X.

1.2 Draw the state diagram/table

- Let X = input, Z = output
- Assume a starting or reset state, S0
- At state S0 the output Z = 0, meaning no previous X=1 triggering condition has occurred yet.

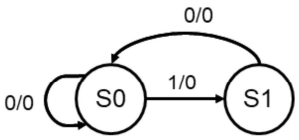


Present State	Next State / Z	
	X = 0	X = 1
S0	S0/0	

In state S0, if the input is 0, the output is also 0, and the circuit stays in S0. If the input is 1, the output remains 0 because it's only the first occurrence—we haven't seen any consecutive 1s yet. However, since it is a 1, we move to the next state to begin tracking the sequence.

1.2 Draw the state diagram/table

- Let X = input, Z = output
- Assume a starting or reset state, S0
- At state S0 the output Z = 0, meaning no previous X=1 triggering condition has occurred yet.
- Let the circuit be in state S1 when the first "1" is detected.

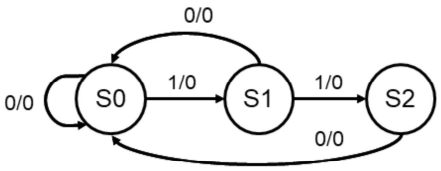


Present State	Next State / Z	
	X = 0	X = 1
S0	S0/0	S1/0
S1	S0/0	

Moving on to state S1, this represents that the first input 1 has been detected. If the next input is 0, the output remains 0, and the circuit returns to S0 to restart the sequence. If the input is 1, the output is still 0 since it's only the second consecutive 1, but the circuit advances to the next state to continue tracking the sequence.

1.2 Draw the state diagram/table

- Let X = input, Z = output
- Assume a starting or reset state, S0
- At state S0 the output Z = 0, meaning no previous X=1 triggering condition has occurred yet.
- Let the circuit be in state S1 when the first "1" is detected.
- Let the circuit be in state S2 when the second consecutive "1" is detected



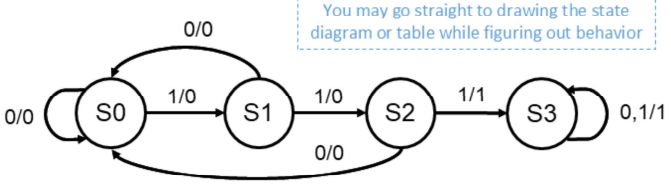
Present State	Next State / Z	
	X = 0	X = 1
S0	S0/0	S1/0
S1	S0/0	S2/0
S2	S0/0	

Next, we move to state S2. In this state, if the input is 0, the output remains 0 and the circuit returns to S0, as the sequence of three consecutive 1s has been broken. However, if the input is 1, this marks the third consecutive 1—so the output becomes 1, and the circuit transitions to the next state.

1.2 Draw the state diagram/table

- Let X = input, Z = output
- Assume a starting or reset state, S0
- At state S0 the output Z = 0, meaning no previous X=1 triggering condition has occurred yet.
- Let the circuit be in state S1 when the first "1" is detected.
- Let the circuit be in state S2 when the second consecutive "1" is detected
- Let the circuit be in state S3 when the third consecutive "1" is detected

You may go straight to drawing the state diagram or table while figuring out behavior



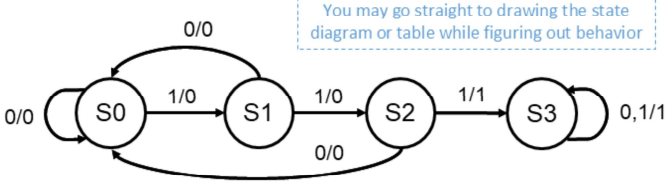
Present State	Next State / Z	
	X = 0	X = 1
S0	S0/0	S1/0
S1	S0/0	S2/0
S2	S0/0	S3/1
S3	S3/1	S3/1

Now, in state S3—where the third consecutive 1 has been detected—the condition of having three consecutive 1s is satisfied. From this point on, the output will remain permanently at 1, regardless of the input value.

1.2 Draw the state diagram/table

- Let X = input, Z = output
- Assume a starting or reset state, S0
- At state S0 the output Z = 0, meaning no previous X=1 triggering condition has occurred yet.
- Let the circuit be in state S1 when the first "1" is detected.
- Let the circuit be in state S2 when the second consecutive "1" is detected
- Let the circuit be in state S3 when the third consecutive "1" is detected

You may go straight to drawing the state diagram or table while figuring out behavior



Present State	Next State / Z	
	X = 0	X = 1
S0	S0/0	S1/0
S1	S0/0	S2/0
S2	S0/0	S3/1
S3	S3/1	S3/1

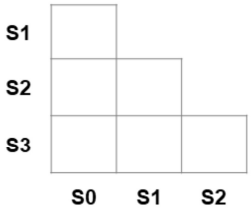
Something to think about: This is a Mealy machine.
What would the state diagram look like if we needed a Moore machine?

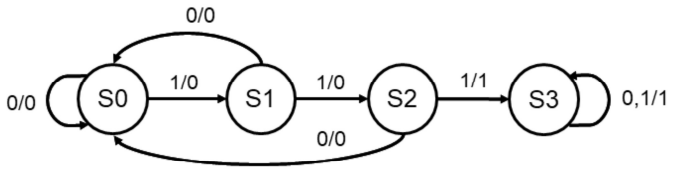
What we just designed is a **Mealy machine**, where the **output depends on both the current state and the input**. This allows the output to change immediately in response to an input, which is why the circuit outputs 1 as soon as the third consecutive 1 is detected.

If we were to design a **Moore machine**, the output would depend **only on the current state**, not on the input. This means the output can't react instantly to an input change—it only updates after the circuit transitions to a new state. So, to achieve the same behavior, we'd need to add an extra state (let's call it **S4**) where the output is 1. This state would be entered **after** detecting the third consecutive 1 in S3.

In summary, a Moore machine version would require **one additional state** compared to the Mealy version and would produce the output **one clock cycle later**, since the output is tied only to the state itself.

1.3 Minimize states



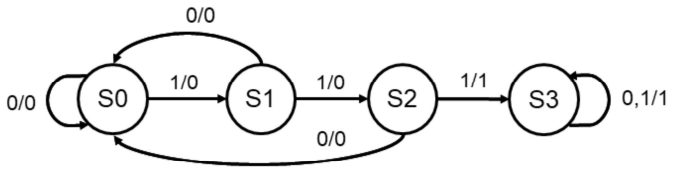


Present State	Next State / Z	
	X = 0	X = 1
S0	S0/0	S1/0
S1	S0/0	S2/0
S2	S0/0	S3/1
S3	S3/1	S3/1

Now, let's move on to the next step: **minimizing the states** based on the state table. To do this, we'll start by creating a **state implication table**, as shown on this slide. This table helps us identify which states can be considered equivalent.

1.3 Minimize states

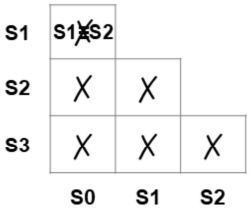
S1	S1≡S2		
S2	X	X	
S3	X	X	X
	S0	S1	S2

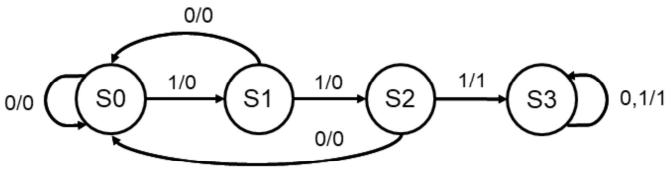


Present State	Next State / Z	
	X = 0	X = 1
S0	S0/0	S1/0
S1	S0/0	S2/0
S2	S0/0	S3/1
S3	S3/1	S3/1

In this case, the only possible pair of equivalent states is **S0 and S1**. For **S0 and S1** to be equivalent, the states they transition to—**S1 and S2**, respectively—must also be equivalent. However, since **S1 and S2 are not equivalent**, this condition is not satisfied. Therefore, **S0 and S1 cannot be considered equivalent either**.

1.3 Minimize states





Present State	Next State / Z	
	X = 0	X = 1
S0	S0/0	S1/0
S1	S0/0	S2/0
S2	S0/0	S3/1
S3	S3/1	S3/1

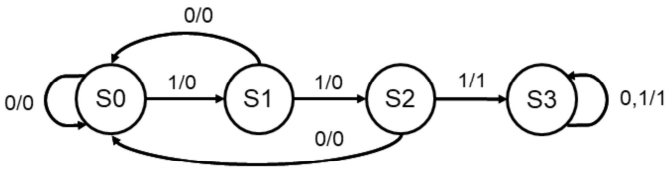
- **Cannot be simplified further**

Hence, the table is already in its simplest form—no further state reduction is possible. With that, we can now proceed to the next step in the design process.

1.4 Assign state variables

Using straight binary assignment, we need 2 state variables to represent 4 states:

- S0 = 00
- S1 = 01
- S2 = 10
- S3 = 11



Present State	Next State / Z	
	X = 0	X = 1
S0	S0/0	S1/0
S1	S0/0	S2/0
S2	S0/0	S3/1
S3	S3/1	S3/1

Something to think about:
Is there a *better* way to assign states?

In this process, we are assigning state variables to represent four distinct states. Since each state requires a unique binary code, and we have four states, we need two binary digits (bits) to cover all possibilities. Using straight binary assignment, we assign the following:

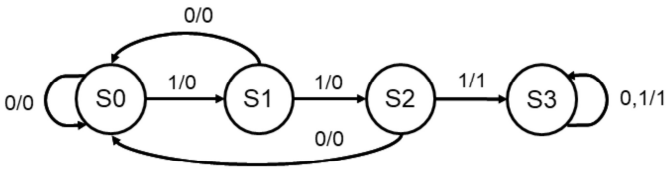
- S0 = 00
- S1 = 01
- S2 = 10
- S3 = 11

This ensures that each state is uniquely identified by a 2-bit binary value, where the binary number corresponds directly to the state in the sequence. Thus, we have effectively used two state variables to represent four distinct states.

1.4 Assign state variables

Using straight binary assignment, we need 2 state variables to represent 4 states:

- S0 = 00
- S1 = 01
- S2 = 10
- S3 = 11



Present State	Next State / Z	
	X = 0	X = 1
S0	S0/0	S1/0
S1	S0/0	S2/0
S2	S0/0	S3/1
S3	S3/1	S3/1

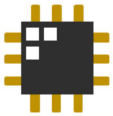
Something to think about:

Is there a *better* way to assign states?

No... hard to guarantee a 1-bit change per state transition all the time while maintaining the same number of bits per state

Is there a more efficient way to assign states?

Generally, no. Achieving a 1-bit change for each state transition while maintaining a consistent number of bits per state is a challenging task. If we try to minimize the number of bit changes (e.g., ensuring only one bit flips during each transition), we would likely need to resort to methods like Gray coding. However, even Gray coding requires careful consideration of state transitions, and it may not always guarantee the same number of bits per state when compared to straightforward binary encoding. Therefore, in most cases, the simplest and most reliable approach remains using direct binary assignment, where each state is represented by a unique binary code, even if this doesn't always provide a 1-bit transition per state change.



1.5 Choose flip-flops

- Positive edge-triggered D flip-flops were specified for this problem
- We take two state variables, Q1 and Q0, from the outputs of two DFFs
  - These correspond to the 2-bit state
- So, we would need the excitation functions D1 and D0, i.e. input of each flip-flop, which are functions of input X and present state Q1, Q0
  1. D1 = f(X, Q1, Q0)
  2. D0 = f (X,Q1, Q0)

Present State	Next State / Z	
	X = 0	X = 1
S0	S0/0	S1/0
S1	S0/0	S2/0
S2	S0/0	S3/1
S3	S3/1	S3/1

Now, let's proceed to selecting the type of flip-flops for our design. According to the problem statement, we are to use positive edge-triggered D flip-flops (DFFs). Since we've assigned two state variables to represent our four states, we will need two D flip-flops—one for each state variable.

Let’s denote the outputs of these flip-flops as Q1 and Q0. These outputs together form the 2-bit code representing the current state of the system.

To determine how the system transitions from one state to another, we must define the excitation functions for the D flip-flops—namely, D1 and D0. These are the inputs to the D flip-flops and determine the next state based on the current input and present state.

Mathematically, these are expressed as:

- D1 = f(X, Q1, Q0)
- D0 = f(X, Q1, Q0)

In other words, the next state (i.e., what we load into the flip-flops) is a function of the current input **X** and the current state represented by **Q1** and **Q0**. These

excitation functions will be crucial for constructing the next state logic.

1.6 Construct excitation table

Excitation Table

Input x	Present State		Next State		Excitation inputs		Output
	$Q1^n$	$Q0^n$	$Q1^{n+1}$	$Q0^{n+1}$	D1	D0	Z

State Table

Present State	Next State / Z	
	X = 0	X = 1
S0	S0/0	S1/0
S1	S0/0	S2/0
S2	S0/0	S3/1
S3	S3/1	S3/1

We will now proceed to construct the **excitation table**, which outlines how the state variables should change based on the current state and input.

1.6 Construct excitation table

Excitation Table

Input x	Present State		Next State		Excitation inputs		Output
	$Q1^n$	$Q0^n$	$Q1^{n+1}$	$Q0^{n+1}$	D1	D0	Z
0	0	0					
0	0	1					
0	1	0					
0	1	1					
1	0	0					
1	0	1					
1	1	0					
1	1	1					

All combinations

State Table

Present State	Next State / Z	
	X = 0	X = 1
S0	S0/0	S1/0
S1	S0/0	S2/0
S2	S0/0	S3/1
S3	S3/1	S3/1

To begin constructing the excitation table, we list all possible combinations of the input **X** and the present state variables **Q1** and **Q0**.

1.6 Construct excitation table

Excitation Table

Input x	Present State		Next State		Excitation inputs		Output
	$Q1^n$	$Q0^n$	$Q1^{n+1}$	$Q0^{n+1}$	D1	D0	Z
0	0	0					
0	0	1					
0	1	0					
0	1	1					
1	0	0					
1	0	1					
1	1	0					
1	1	1					

Determine from state table and state assignment

State Table

Present State	Next State / Z	
	X = 0	X = 1
S0	S0/0	S1/0
S1	S0/0	S2/0
S2	S0/0	S3/1
S3	S3/1	S3/1

State Assignment

S0 = 00

S1 = 01

S2 = 10

S3 = 11

The **next state** and **output** columns of the excitation table are derived based on the **state table** and the **state assignments** we established earlier. By referencing how each current state transitions to the next state for a given input, as defined in the state table, and translating those states into their corresponding binary codes from our state assignment, we can accurately fill in these columns. This step ensures the excitation table reflects the correct behavior of the system for every possible input and present state combination.

1.6 Construct excitation table

Excitation Table

Input x	Present State		Next State		Excitation inputs		Output
	$Q1^n$	$Q0^n$	$Q1^{n+1}$	$Q0^{n+1}$	D1	D0	Z
0	0	0					
0	0	1					
0	1	0	0	0			0
0	1	1					
1	0	0					
1	0	1					
1	1	0					
1	1	1					

Determine from state table and state assignment

When X = 0 and the present state is S2 (10), the next state is S0 (00) and the output is 0

State Table

Present State	Next State / Z	
	X = 0	X = 1
S0	S0/0	S1/0
S1	S0/0	S2/0
S2	S0/0	S3/1
S3	S3/1	S3/1

State Assignment
S0 = 00
S1 = 01
S2 = 10
S3 = 11

For example, consider the case where the present state is **Q1 = 1** and **Q0 = 0**, and the input **X = 0**. According to our state assignment, this corresponds to **state S2**. Referring to the state table, we see that when the system is in S2 and the input is 0, the next state is **S0**, which is represented by **00**, and the output is **0**. This information allows us to fill in the corresponding row of the excitation table for that specific combination of input and present state.

1.6 Construct excitation table

Excitation Table

Input x	Present State		Next State		Excitation inputs		Output
	$Q1^n$	$Q0^n$	$Q1^{n+1}$	$Q0^{n+1}$	D1	D0	Z
0	0	0	0	0			0
0	0	1	0	0			0
0	1	0	0	0			0
0	1	1	1	1			1
1	0	0	0	1			0
1	0	1	1	0			0
1	1	0	1	1			1
1	1	1	1	1			1

Determine from state table and state assignment

State Table

Present State	Next State / Z	
	X = 0	X = 1
S0	S0/0	S1/0
S1	S0/0	S2/0
S2	S0/0	S3/1
S3	S3/1	S3/1

State Assignment
S0 = 00
S1 = 01
S2 = 10
S3 = 11

Now, let's complete the excitation table together by going through each combination of input and present state. We'll use our state assignments and the state transition table to determine the corresponding next state and output for each case:

- For **X = 0**, **Q1 = 0**, **Q0 = 0** (S0): the next state is **S0 (00)**, and the output is **0**.
- For **X = 0**, **Q1 = 0**, **Q0 = 1** (S1): the next state is **S0 (00)**, and the output is **0**.
- For **X = 0**, **Q1 = 1**, **Q0 = 0** (S2): the next state is **S0 (00)**, and the output is **0**.
- For **X = 0**, **Q1 = 1**, **Q0 = 1** (S3): the next state is **S3 (11)**, and the output is **1**.

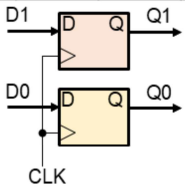
Now for the cases when **X = 1**:

- For **X = 1**, **Q1 = 0**, **Q0 = 0** (S0): the next state is **S1 (01)**, and the output is **0**.
- For **X = 1**, **Q1 = 0**, **Q0 = 1** (S1): the next state is **S2 (10)**, and the output is **0**.
- For **X = 1**, **Q1 = 1**, **Q0 = 0** (S2): the next state is **S3 (11)**, and the output is **1**.
- For **X = 1**, **Q1 = 1**, **Q0 = 1** (S3): the next state remains **S3 (11)**, and the output is **1**.

1.6 Construct excitation table

Excitation Table

Input x	Present State		Next State		Excitation inputs		Output
	$Q1^n$	$Q0^n$	$Q1^{n+1}$	$Q0^{n+1}$	D1	D0	Z
0	0	0	0	0			0
0	0	1	0	0			0
0	1	0	0	0			0
0	1	1	1	1			1
1	0	0	0	1			0
1	0	1	1	0			0
1	1	0	1	1			1
1	1	1	1	1			1



How to get next states given the present state?

Use FF excitation table

State Table

Present State	Next State / Z	
	X = 0	X = 1
S0	S0/0	S1/0
S1	S0/0	S2/0
S2	S0/0	S3/1
S3	S3/1	S3/1

Excitation table for a D flipflop

Given the current state and the desired next state, what should the input(s) be to achieve that?

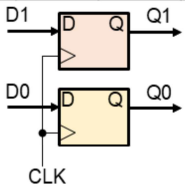
Q_n	Q_{n+1}	D
0	0	0
0	1	1
1	0	0
1	1	1

Now, let's move on to filling in the **excitation inputs** columns, specifically **D1** and **D0**. To do this, we refer to the characteristic behavior of a D flip-flop—remember, for a D flip-flop, the input simply takes on the value of the desired next state at the next clock edge.

1.6 Construct excitation table

Excitation Table

Input x	Present State		Next State		Excitation inputs		Output
	$Q1^n$	$Q0^n$	$Q1^{n+1}$	$Q0^{n+1}$	D1	D0	Z
0	0	0	0	0	0	0	0
0	0	1	0	0	0	0	0
0	1	0	0	0	0	0	0
0	1	1	1	1	1	1	1
1	0	0	0	1	0	1	0
1	0	1	1	0	1	0	0
1	1	0	1	1	1	1	1
1	1	1	1	1	1	1	1



How to get next states given the present state?

Use FF excitation table

State Table

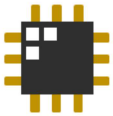
Present State	Next State / Z	
	X = 0	X = 1
S0	S0/0	S1/0
S1	S0/0	S2/0
S2	S0/0	S3/1
S3	S3/1	S3/1

Excitation table for a D flipflop

Given the current state and the desired next state, what should the input(s) be to achieve that?

Q_n	Q_{n+1}	D
0	0	0
0	1	1
1	0	0
1	1	1

So, we directly copy the values from the **next state** columns. In other words, the value of **D1** will be the next value of **Q1**, and **D0** will be the next value of **Q0**, as determined in the previous step.



1.6 Construct excitation table

Excitation Table

Input x	Present State		Next State		Excitation inputs		Output
	$Q1^n$	$Q0^n$	$Q1^{n+1}$	$Q0^{n+1}$	D1	D0	Z
0	0	0	0	0	0	0	0
0	0	1	0	0	0	0	0
0	1	0	0	0	0	0	0
0	1	1	1	1	1	1	1
1	0	0	0	1	0	1	0
1	0	1	1	0	1	0	0
1	1	0	1	1	1	1	1
1	1	1	1	1	1	1	1

With the excitation table now complete, we're ready to move on to the next step in the design process.

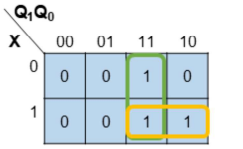
1.7 Derive next-state and output functions

Write Boolean expressions for the flipflop inputs (D1 and D0) and the circuit output (Z)

Input x	Present State		Next State		Excitation inputs		Output
	$Q1^n$	$Q0^n$	$Q1^{n+1}$	$Q0^{n+1}$	D1	D0	Z
0	0	0	0	0	0	0	0
0	0	1	0	0	0	0	0
0	1	0	0	0	0	0	0
0	1	1	1	1	1	1	1
1	0	0	0	1	0	1	0
1	0	1	1	0	1	0	0
1	1	0	1	1	1	1	1
1	1	1	1	1	1	1	1

K-map inputs

One K-map for each of these



$Z = XQ_1 + Q_1Q_0$

Now, let's move on to **deriving the next-state and output functions** by simplifying the flip-flop input equations and the output using **Karnaugh maps (K-maps)**.

We'll begin with the **output function**. As shown on the right, we've filled out the K-map based on the output values from the state table. By grouping the 1s according to the K-map rules—forming the largest possible groups of 1, 2, or 4 adjacent cells—we can simplify the Boolean expression.

From this, we get the output function:
Z = XQ1 + Q1Q0

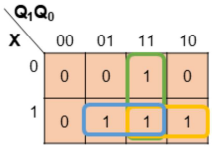
1.7 Derive next-state and output functions

Write Boolean expressions for the flipflop inputs (D1 and D0) and the circuit output (Z)

Input x	Present State		Next State		Excitation inputs		Output
	$Q1^n$	$Q0^n$	$Q1^{n+1}$	$Q0^{n+1}$	D1	D0	Z
0	0	0	0	0	0	0	0
0	0	1	0	0	0	0	0
0	1	0	0	0	0	0	0
0	1	1	1	1	1	1	1
1	0	0	0	1	0	1	0
1	0	1	1	0	1	0	0
1	1	0	1	1	1	1	1
1	1	1	1	1	1	1	1

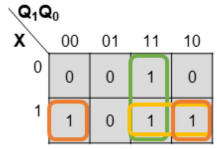
K-map inputs

One K-map for each of these



$D_1 = XQ_0 + XQ_1 + Q_1Q_0$

$D_1 = XQ_0 + Z$



$D_0 = XQ_0' + XQ_1 + Q_1Q_0$

$D_0 = XQ_0' + Z$

Now, let's proceed with the Karnaugh maps for the flip-flop inputs **D1** and **D0**, as shown on the right side of the slide.

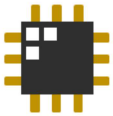
Starting with **D1**, we derived the initial expression:
D1 = XQ0 + XQ1 + Q1Q0.

Notice that the last two terms—**XQ1 + Q1Q0**—match the output function we previously simplified as **Z**. By substituting this into the equation, we can simplify D1 to:
D1 = XQ0 + Z.

Next, for **D0**, the original expression is:
D0 = XQ0' + XQ1 + Q1Q0.

Again, the last two terms form the output **Z**, so we can simplify this to:
D0 = XQ0' + Z.

By recognizing and substituting the output expression **Z**, we reduce both D1 and D0 to more compact forms, which helps simplify the final circuit.



1.7 Derive next-state and output functions

Write Boolean expressions for the flipflop inputs (D1 and D0) and the circuit output (Z)

Input x	Present State		Next State		Excitation inputs		Output
	$Q1^n$	$Q0^n$	$Q1^{n+1}$	$Q0^{n+1}$	D1	D0	Z
0	0	0	0	0	0	0	0
0	0	1	0	0	0	0	0
0	1	0	0	0	0	0	0
0	1	1	1	1	1	1	1
1	0	0	0	1	0	1	0
1	0	1	1	0	1	0	0
1	1	0	1	1	1	1	1
1	1	1	1	1	1	1	1

$Z = XQ_1 + Q_1Q_0$

$D_1 = XQ_0 + XQ_1 + Q_1Q_0$

$D_0 = XQ_0' + XQ_1 + Q_1Q_0$

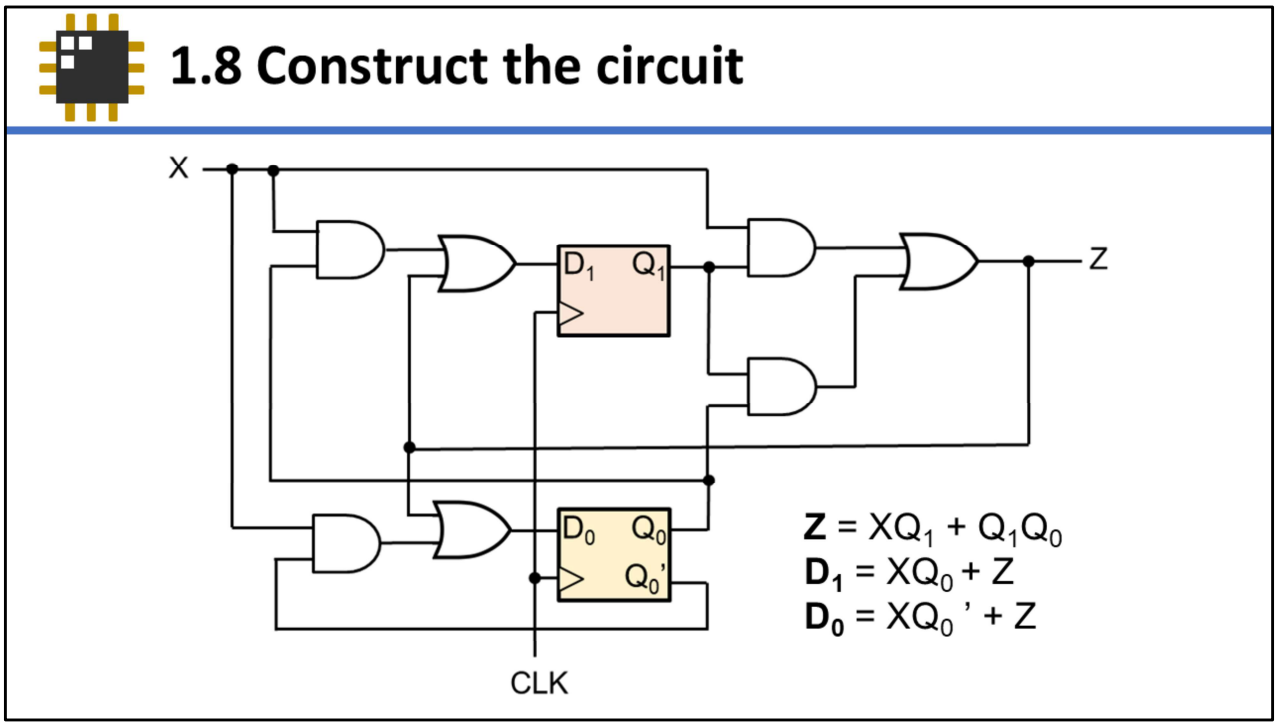
Simpler version:

$Z = XQ_1 + Q_1Q_0$

$D_1 = XQ_0 + Z$

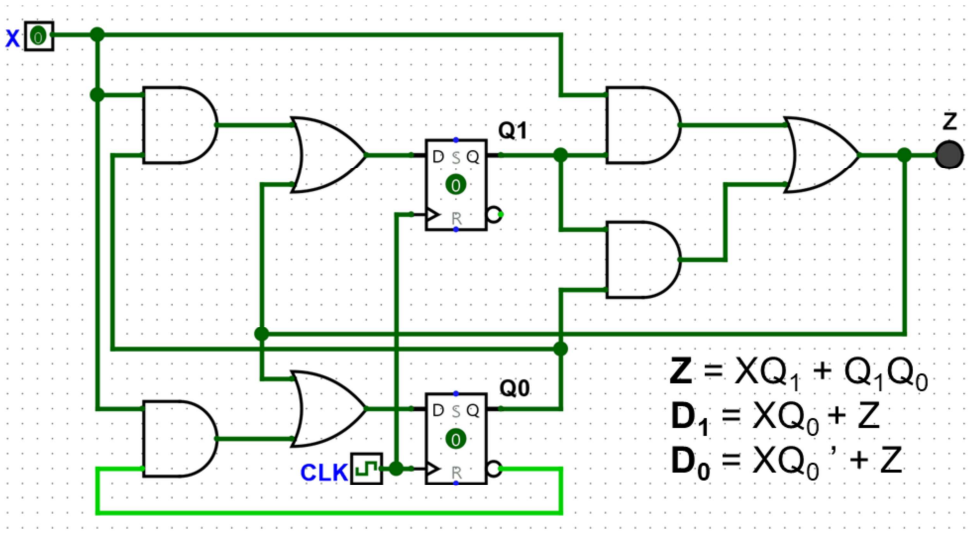
$D_0 = XQ_0' + Z$

Now that we've derived the next-state and output functions, we can use these simplified equations to **construct the logic circuit**. These expressions will guide us in connecting the logic gates and flip-flops needed to implement the desired state machine behavior.



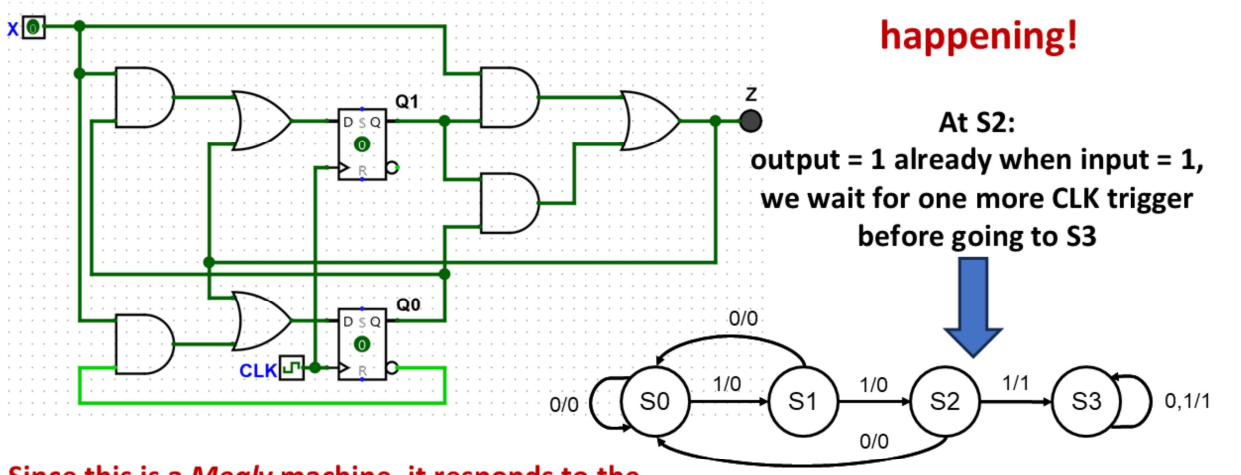
Here's the constructed circuit based on the next-state and output equations.

1.9 (Bonus) Simulate and verify!



To verify our design, here's a GIF showing the implemented circuit in action with different input sequences. This demonstrates how the circuit responds.

1.9 (Bonus) Simulate and verify!



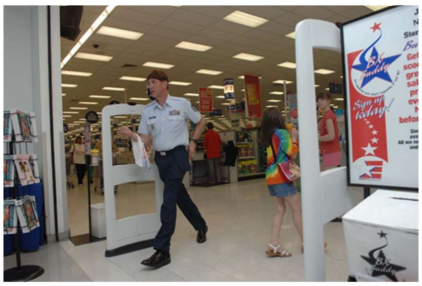
When the system is in state S2, notice that the output is already 1, even before the clock signal triggers or before the system transitions to state S3. This behavior is characteristic of a Mealy machine, where the output depends not only on the current state but also on the current input. In this case, the input condition for an output of 1 is already satisfied while the system is still in S2, which is why we see the output change immediately rather than waiting for the next state transition.

Example 2

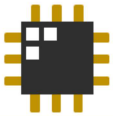
Shop-lifting Alarm:

Design a shop-lifting alarm that is triggered when items are brought out the door without being purchased.

- A sensor **S** becomes 1 if such an attempt is made, and the alarm **A** sounds.
- The clerk has a switch **R** that resets the alarm after the item is brought back to the store.



Let's move on to the next example, the shop-lifting alarm.



2.1 Describe behavior/function

- S = 1 makes A = 1
- R = 1 makes A = 0 if S is now 0
- Two states: OK and BEEP

This system has two states: idle and alarmed.

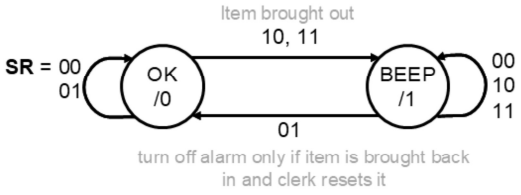
During normal conditions, S = 0, meaning there is no attempt to remove an item improperly. So, the alarm A = 0.

When S = 1, that means the system has detected potential theft. A should be 1 immediately.

Once the situation has been handled, the clerk can press R = 1 to reset the system. This causes A = 0 if S = 0.

2.2 Draw the state diagram/table

- S = 1 makes A = 1
- R = 1 makes A = 0 if S is now 0
- Two states: OK and BEEP



Present State	Next State				A
	SR = 00	01	10	11	
OK	OK	OK	BEEP	BEEP	0
BEEP	BEEP	OK	BEEP	BEEP	1

Now, let's proceed to constructing the state diagram or state table for this system. Since we're working with two inputs—**S** (sensor) and **R** (reset)—we have a total of four possible input combinations to consider: 00, 01, 10, and 11.

We'll start with the **initial or idle state**, which we'll refer to as the **OK state**. In this state, the alarm has not been triggered, so the output **A = 0**. Regardless of the input combination, the output will remain 0 in this state.

Let's analyze the inputs in the OK state:

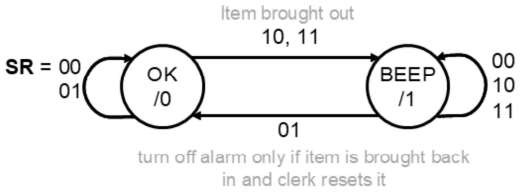
- For **SR = 00** and **SR = 01**, the system stays in the OK state because the sensor input **S** is still 0—meaning no unauthorized removal has been detected.
- However, for **SR = 10** and **SR = 11**, the sensor detects **S = 1**, indicating a possible theft. As a result, the system transitions to the **BEEP state**, and the alarm output becomes **A = 1**.

Now, let's move on to the **BEEP state**, where the alarm is actively sounding (**A = 1**). In this state, we're looking for a reset signal from the clerk to silence the alarm.

- Among all possible input combinations, only **SR = 01** (S = 0, R = 1) satisfies the condition for deactivating the alarm. This means that the sensor no longer detects an issue, and the clerk has pressed the reset button. Therefore, the system transitions back to the OK state, and the alarm is turned off.
- For the remaining inputs—**SR = 00**, **10**, and **11**—the system stays in the BEEP state because either the sensor still detects a problem, the reset hasn't been triggered, or both.

2.3-2.5 Minimize; Assign; Choose flipflops

- Cannot be minimized further
- Two states = one state variable Q
  - OK = 0
  - BEEP =1
- Choose D flip-flops for now

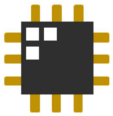


Present State	Next State				A
	SR = 00	01	10	11	
0	0	0	1	1	0
1	1	0	1	1	1

Let's now move on to the **state minimization** process. In this case, the system only consists of two states—**OK** and **BEEP**. Since these two states serve distinct purposes and have different behaviors (one indicating normal operation and the other indicating an active alarm), they are **not equivalent**. Therefore, no further simplification or merging of states is possible.

With that settled, we can proceed to **assign binary values to the states**, which is a necessary step for implementing the design in hardware. Since the **OK state** is our starting or initial state, we'll assign it a state variable of **0**. The **BEEP state**, which is entered when the alarm is triggered, will be assigned the state variable **1**.

Next, we need to decide on the type of **flip-flop** to use for storing the state information. For this design, we'll use a **D flip-flop**, which is a popular choice due to its straightforward behavior—it simply transfers the input (D) to the output (Q) on the rising edge of the clock. This makes it easier to derive the logic for the next state, as the output directly reflects the value of the D input.



2.6 Construct excitation table

Inputs		Present State	Next State	Excitation	Output
S	R	Q^n	Q^{n+1}	D	A
0	0	0			
0	0	1			
0	1	0			
0	1	1			
1	0	0			
1	0	1			
1	1	0			
1	1	1			

All combinations

Present State	Next State				A
	SR = 00	01	10	11	
0	0	0	1	1	0
1	1	0	1	1	1

What should be next state and output be?

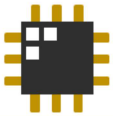
Now, let's move on to constructing the **excitation table**. This table helps us determine the logic needed to drive the flip-flop input based on the current state and the inputs. To begin, we'll list **all possible combinations of the inputs (S and R)** along with the **present state**. From these, we'll derive the **next state** and the **output**, using the information we've already established from the state table.

We'll go through the table row by row together:

- For **SR = 00** and **present state = 0 (OK)**, the system remains in state 0, and the output is 0.
- For **SR = 00** and **present state = 1 (BEEP)**, the system stays in state 1, and the output is 1.
- For **SR = 01** and **present state = 0**, the next state is still 0, and the output remains 0.
- For **SR = 01** and **present state = 1**, the system transitions to state 0, as the reset is triggered, and the output is 1.
- For **SR = 10** and **present state = 0**, the system detects a shop-lifting attempt and moves to state 1. The output remains 0 during the transition.
- For **SR = 10** and **present state = 1**, the system remains in state 1, and the

output stays at 1.

- For **SR = 11** and **present state = 0**, the sensor input is 1, so the system transitions to state 1. The output remains 0 during this transition.
- For **SR = 11** and **present state = 1**, the system stays in state 1, with the output continuing to be 1.



2.6 Construct excitation table

Inputs		Present State	Next State	Excitation	Output
S	R	Q^n	Q^{n+1}	D	A
0	0	0	0		0
0	0	1	1		1
0	1	0	0		0
0	1	1	0		1
1	0	0	1		0
1	0	1	1		1
1	1	0	1		0
1	1	1	1		1

All combinations

Present State	Next State				A
	SR = 00	01	10	11	
0	0	0	1	1	0
1	1	0	1	1	1

What should the flipflop input be to reach the desired next state?

Now, let's proceed to determining the **excitation for the flip-flop**. Since we are using a **D flip-flop**, this step is quite straightforward. Recall that the D flip-flop transfers the input directly to the output on the clock edge. This means that the **D input should match the desired next state** of the system. So, in constructing the excitation table, we simply **copy the values from the next state column** into the D input column.

2.6 Construct excitation table

Inputs		Present State	Next State	Excitation	Output
S	R	Q^n	Q^{n+1}	D	A
0	0	0	0	0	0
0	0	1	1	1	1
0	1	0	0	0	0
0	1	1	0	0	1
1	0	0	1	1	0
1	0	1	1	1	1
1	1	0	1	1	0
1	1	1	1	1	1

All combinations

Present State	Next State				A
	SR = 00	01	10	11	
0	0	0	1	1	0
1	1	0	1	1	1

Now that we've completed the excitation table, we're ready to move on to the next step.

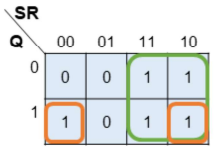
2.7 Derive next-state and output functions

Inputs		Present State	Next State	Excitation	Output
S	R	Q^n	Q^{n+1}	D	A
0	0	0	0	0	0
0	0	1	1	1	1
0	1	0	0	0	0
0	1	1	0	0	1
1	0	0	1	1	0
1	0	1	1	1	1
1	1	0	1	1	0
1	1	1	1	1	1

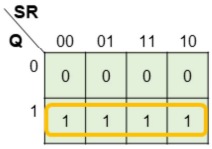
All combinations

Present State	Next State SR = 00	01	10	11	A
0	0	0	1	1	0
1	1	0	1	1	1

Using K-maps to derive the Boolean expressions



D = S + QR'

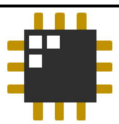


A = Q

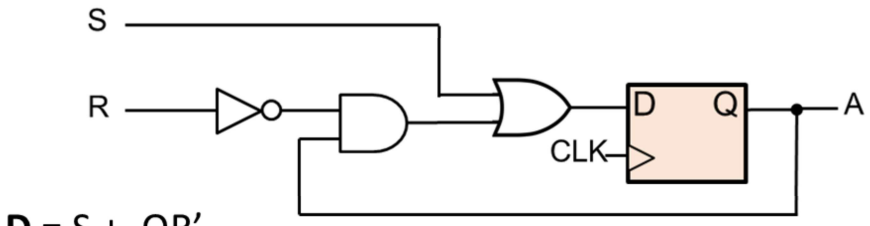
Now, let's derive the **next-state** and **output functions** by simplifying the expressions for the flip-flop input and the output using **Karnaugh maps (K-maps)**. Remember, when simplifying with K-maps, the goal is to form the **largest possible groups** of 1, 2, 4, or even 8 adjacent cells containing 1s, in order to minimize the logic expression.

You can refer to the K-maps shown on the bottom left for a visual guide. Based on the K-map simplification, we arrive at the following expressions:

- **D = S + Q·R'** → This determines the next state of the system.
- **A = Q** → This directly defines the output, indicating that the alarm is active when the system is in the BEEP state.



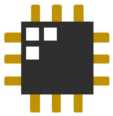
2.8 Construct the circuit



D = S + QR’

A = Q

Here’s the constructed circuit based on the next-state and output equations.



What if we use a JKFF for Example 2?

Inputs		Present State	Next State	Excitation		Output
S	R	Q^n	Q^{n+1}	J	K	A
0	0	0	0	0	x	0
0	0	1	1	x	0	1
0	1	0	0	0	x	0
0	1	1	0	x	1	1
1	0	0	1	1	x	0
1	0	1	1	x	0	1
1	1	0	1	1	x	0
1	1	1	1	x	0	1

Present State	Next State				A
	SR = 00	01	10	11	
0	0	0	1	1	0
1	1	0	1	1	1

Excitation table for a JK flipflop

Given the current state and the desired next state, what should the input(s) be to achieve that?

Q_n	Q_{n+1}	J	K
0	**0**	**0**	**X**
0	**1**	**1**	**X**
1	**0**	**X**	**1**
1	**1**	**X**	**0**

Now, let's explore what happens if we use a **JK flip-flop** instead for **Example 2**.

As before, the **excitation values for the flip-flop** will depend on the type of flip-flop used. In the case of a JK flip-flop, we'll need to refer to its **excitation table** to determine the required inputs (J and K) based on the **present state (Q_n)** and the **next state (Q_{n+1})**.

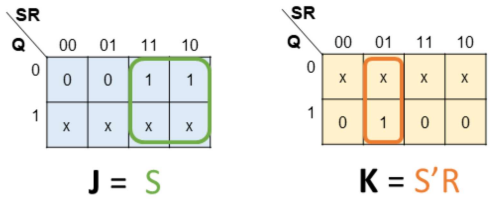
You can refer to the **JK excitation table** shown at the bottom left as we go through the process of filling out our own table. Let's walk through it together, row by row:

- For Q_n **= 0** and Q_{n+1} **= 0**, the required JK input is **0X** (no change).
- For Q_n **= 1** and Q_{n+1} **= 1**, JK is **X0** (no change).
- For Q_n **= 0** and Q_{n+1} **= 0**, again, JK is **0X**.
- For Q_n **= 1** and Q_{n+1} **= 0**, JK must be **X1** (reset).
- For Q_n **= 0** and Q_{n+1} **= 1**, JK is **1X** (set).
- For Q_n **= 1** and Q_{n+1} **= 1**, JK is **X0**.
- For Q_n **= 0** and Q_{n+1} **= 1**, JK is **1X**.
- And finally, for Q_n **= 1** and Q_{n+1} **= 1**, JK remains **X0**.

What if we use a JKFF for Example 2?

Inputs		Present State	Next State	Excitation		Output
S	R	Q^n	Q^{n+1}	J	K	A
0	0	0	0	0	x	0
0	0	1	1	x	0	1
0	1	0	0	0	x	0
0	1	1	0	x	1	1
1	0	0	1	1	x	0
1	0	1	1	x	0	1
1	1	0	1	1	x	0
1	1	1	1	x	0	1

A = Q still, while the FF inputs are:

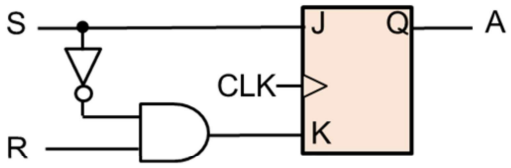


Now, let's derive the **next-state** by simplifying the expressions for the flip-flop input and the output using **Karnaugh maps (K-maps)**. Remember, when simplifying with K-maps, the goal is to form the **largest possible groups** of 1, 2, 4, or even 8 adjacent cells containing 1s, in order to minimize the logic expression.

You can refer to the K-maps shown on the bottom left for a visual guide. Based on the K-map simplification, we arrive at the following expressions:

- **J = S**
- **K = S'R**

What if we use a JKFF for Example 2?

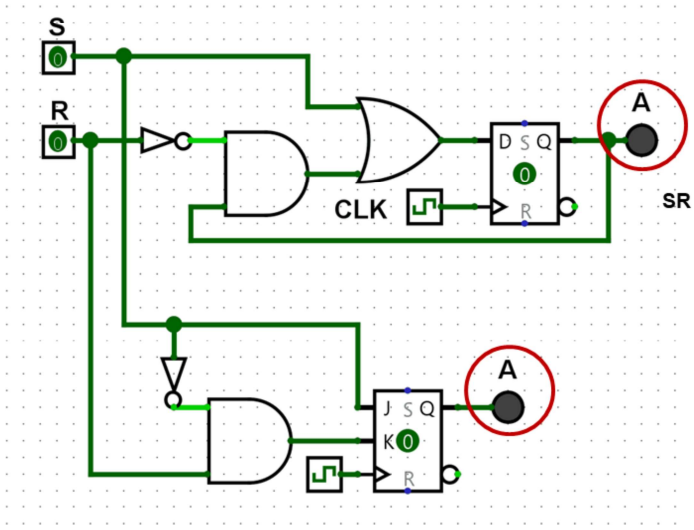


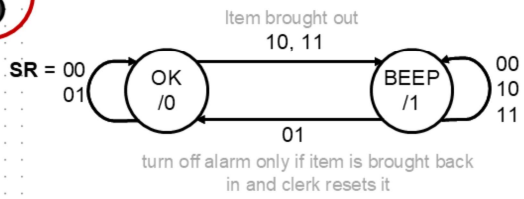
J = S
K = S'R
A = Q

Flipflop choice matters!

Here is the **final circuit** constructed based on the derived **excitation and output equations**. You'll notice that, compared to the version using a D flip-flop, this design requires **one less OR gate**. This highlights an important design consideration: the choice of **flip-flop** can significantly impact the **complexity and efficiency** of the circuit.

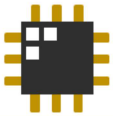
Different FF choice, same state diagram





Present State	Next State				A
	SR = 00	01	10	11	
0	0	0	1	1	0
1	1	0	1	1	1

Selecting the most appropriate flip-flop for a given application can help **simplify the logic**, reduce component count, and ultimately make the implementation more cost-effective and efficient.

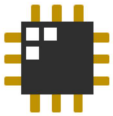


Recap: Synthesis Procedure

1. Describe behavior/function
2. Draw state diagram/table
3. Minimize states
4. Assign state variables
5. Choose flip-flops
6. Construct excitation table
7. Derive next-state and output functions
8. Construct the logic diagram/circuit
9. *Test the circuit (e.g. simulation)*

Recall that the **synthesis procedure** is a step-by-step method used to design a sequential circuit. First, you describe what the circuit should do—its purpose and how the outputs should respond to the inputs. Then, you draw a state diagram or state table to show how the system moves from one state to another based on different input conditions. After that, you try to reduce the number of states by combining any that behave the same, which makes the design simpler. Next, you assign binary codes (1s and 0s) to each state so the circuit can understand them.

Once the states are assigned, you choose the type of flip-flop to use—usually a D or JK flip-flop, depending on what's available or what the problem gives you. Then, you create an excitation table to figure out what input each flip-flop needs to make the correct transitions between states. From this, you write the equations for the next state and the output, using Boolean algebra or Karnaugh maps to make them simpler. After all that, you build the circuit using logic gates and flip-flops based on your equations. Lastly, you test the circuit, either in a simulator or with real hardware, to make sure it works as intended. This process helps you build a correct and efficient circuit.



Recap: Synthesis Procedure

As long as we drew the
correct state diagram,
everything else is methodical!

Translating a word problem into a state diagram correctly is critical

As long as we begin with a **correct state diagram**, everything that follows—such as the state table, excitation table, logic expressions, and final circuit design—becomes a **methodical, step-by-step process**. Each step builds directly on the last, and with the proper framework in place, it becomes much easier to move forward with confidence and accuracy.

Example 3: Train passage signals

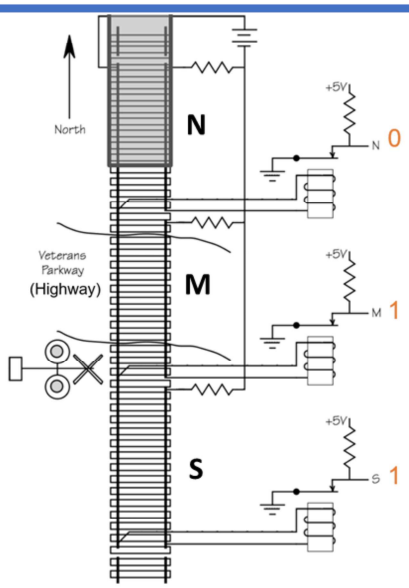
A more complicated example ☺

Screengrab from https://www.youtube.com/watch?v=n2n7XF9YhnE

Now, let's move on to our last and more complicated example, the train passage signals.

Example 3: Train passage signals

- The train progresses through the three sections of the track:
  North (**N**), Middle (**M**), and South (**S**) sections
- If **train is long enough**, the three sections go from NMS = 111 to 011 to 001 to 000 to 100 to 110 to 111 again.
- A **short train** could cause NMS = 111 to 011 to 001 to 101 to 100 to 110 to 111.



In this example, we'll look at a **Train Passage Signal** system—a slightly more complex problem that models how a train moves through different sections of a track and how the system responds.

Imagine that the train track is divided into **three sections**:

- **N** for **North**,
- **M** for **Middle**, and
- **S** for **South**.

Each of these sections is equipped with a **sensor** that detects whether a train is present. These sensors give a binary signal:

- **1** means **no train** is detected in that section,
- **0** means **a train is present** in that section.

So when we write something like NMS = 111, it means all three sections are clear. Conversely, NMS = 000 would mean the train is occupying **all three sections**.

Let's consider two scenarios:

1. Long Train Movement

If the train is **long enough** to occupy all three sections at some point, the sequence of sensor readings might look like this:

111 → 011 → 001 → 000 → 100 → 110 → 111

Here's what's happening:

- **111**: No train yet.
- **011**: The train has entered the **North** section (N = 0).
- **001**: The train is now in the **North and Middle** sections.
- **000**: The train covers **all three sections**.
- **100**: The front of the train has exited the **South** section, but the rear is still in the North and Middle.
- **110**: The train is leaving, only the rear is in the North section.
- **111**: All sections are clear again.

2. Short Train Movement

If the train is **shorter** and cannot occupy all three sections at once, the sensor readings could be:

111 → 011 → 001 → 101 → 100 → 110 → 111

In this case:

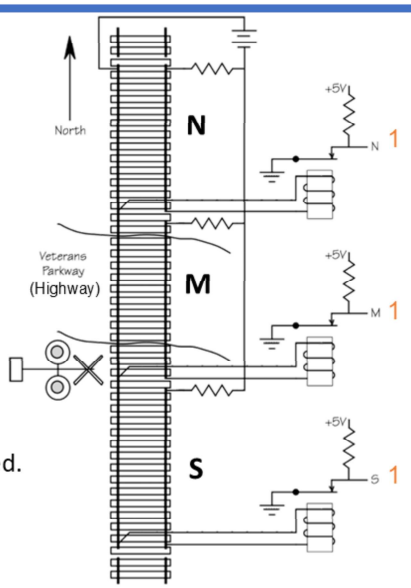
- The train enters and passes through the sections without ever occupying all three simultaneously.
- Notice the appearance of **101**, which means the train is in the **Middle section**, but both **North and South** are clear. This wouldn't be possible with a long train, but it's typical for a short one.

By understanding these sequences, we can start thinking about how the system should react—for example, when to raise or lower barriers, or when to turn signals on or off. Recognizing the patterns in sensor readings allows us to design a state machine that can **track the position and movement of the train**, whether it's short or long.

Example 3: Train passage signals

Draw the state diagram for a circuit that will activate the train passage signal (let's call it **F**):

- When a train enters either N or S section, set **F = 1**.
- **F = 0** when the train is out of the crossing itself (M), thus:
  - A train entering at N section makes **F = 1**.
  - **F = 1** until the train has left both N and M, but it can still be in S.
  - or
  - A train entering at S section makes **F = 1**.
  - **F = 1** until the train has left both S and M, but is can still be in N.
- **F = 0** when there is no train crossing or the train has already crossed.



Now that we understand how a train moves across the North (N), Middle (M), and South (S) sections of the track, let's take a closer look at how to design a state diagram for the **Train Passage Signal**, denoted by **F**. This signal is meant to activate when a train is crossing and deactivate once the train has completely cleared the area.

When should F = 1?

The output **F** should become **1** when a train is detected in either the **North** or **South** section. This tells us that a train is approaching or already entering the crossing. So:

- If a train enters from the **North**, we set **F = 1**.
- If a train enters from the **South**, we also set **F = 1**.

Once activated, **F** should remain **1** while the train is still in the crossing area or approaching it from the direction it came.

When should F return to 0?

The signal should only go back to **0** after the train has fully exited the **Middle section** as well as the section it originally entered from.

- If the train came from the **North**, **F = 0** only after it has left both the **North**

and **Middle** sections. It may still be in the **South**, but the signal can already be deactivated at this point.

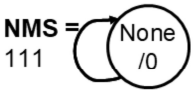
- Similarly, if it came from the **South**, **F = 0** once it has left both **South** and **Middle**.

This ensures that **F stays 1 for the entire duration of the train crossing**, regardless of how long the train is.

When should F stay at 0?

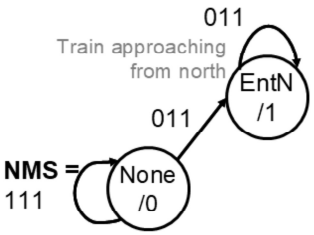
F remains at **0** if there is no train in any section, or if a train has already fully passed through and cleared all relevant parts of the crossing. Essentially, F = 0 means the crossing is safe and clear.

Drawing the state diagram



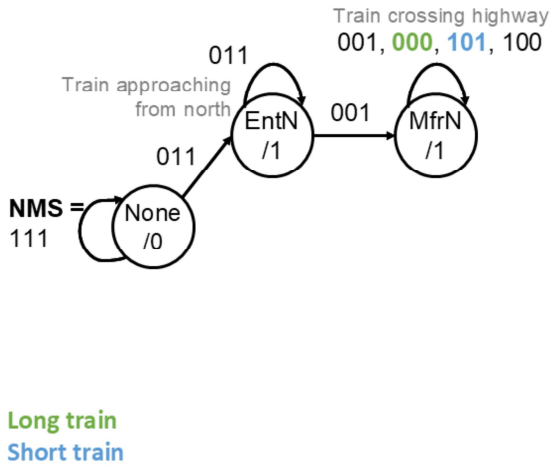
Let's begin with the **initial state**, which represents the scenario where no train is detected in any of the three sections. In this case, the system remains in the None state (NMS = 111)—it transitions back to itself as long as no train is detected. Also note, that F = 0, in this case because there are no trains detected.

Drawing the state diagram



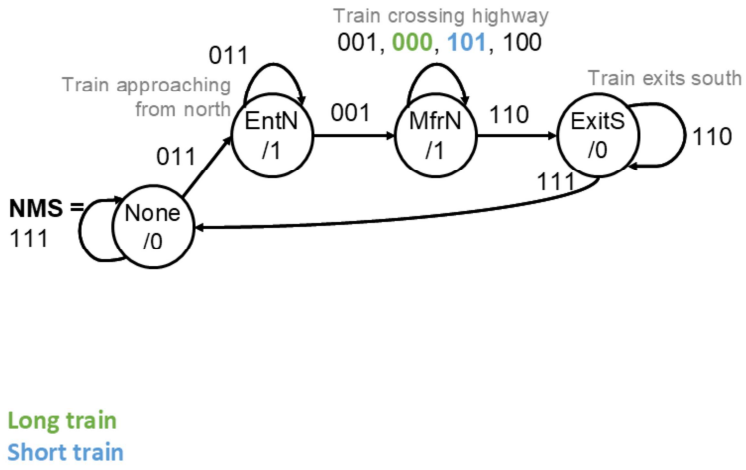
Next, let's move on to the next state: when the train is approaching from the **North** (**NMS = 011**). We'll call this the **EntN** state. From the **None** state, the system transitions to **EntN** when **NMS = 011**, indicating a train is detected and **F = 1** (train enters N).

Drawing the state diagram



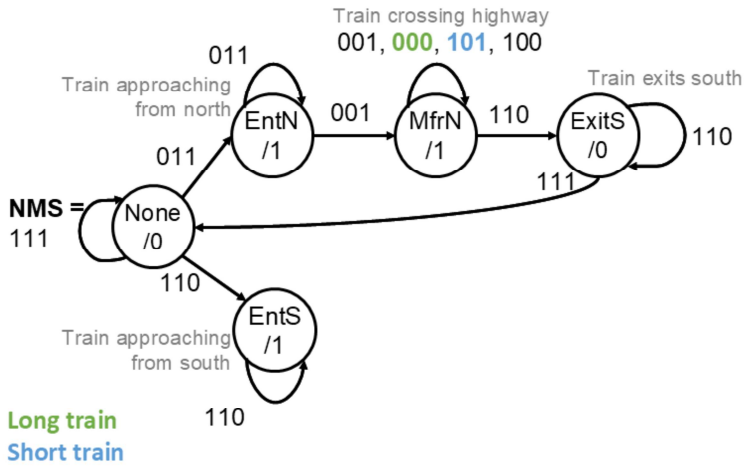
Next, let's consider the state where the train is crossing the highway. This includes the cases where **NMS = 001, 000, 101,** and **100**. From the **EntN** state, the system transitions to the **MfrN** state when **NMS = 001**, indicating that the train is crossing the highway, and **F = 1**.

Drawing the state diagram



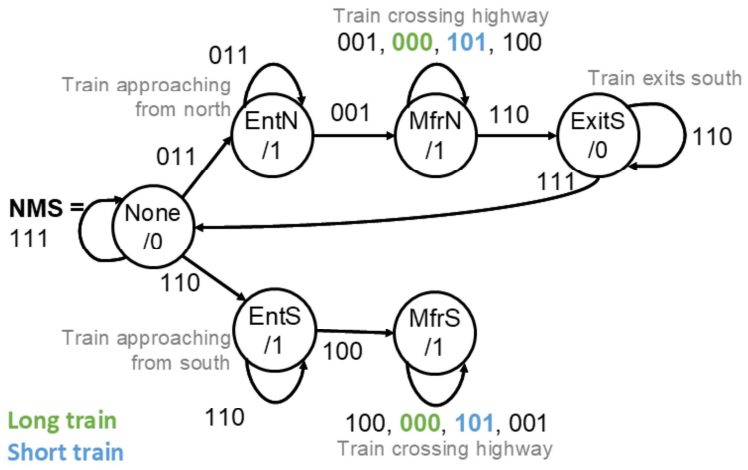
Now, let's consider the state where the train exits the South. We'll call this the **ExitS** state. From the **MfrN** state, the system transitions to **ExitS** when **NMS = 110** and **F = 0**, indicating that the train has exited the crossing but is still in the South section. Once the train is completely out (**NMS = 111**), the system will return to the **None** state.

Drawing the state diagram



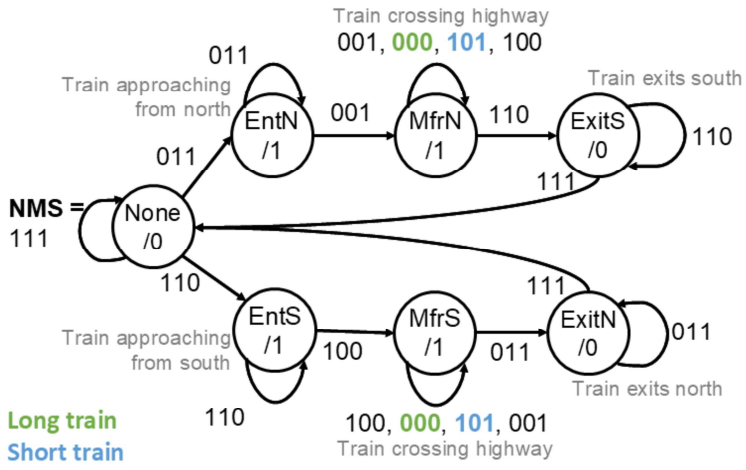
Next, let's move on to the next state: when the train is approaching from the **South** (**NMS = 110**). We'll call this the **EntS** state. From the **None** state, the system transitions to **EntS** when **NMS = 110** and F = 1, indicating a train is detected.

Drawing the state diagram



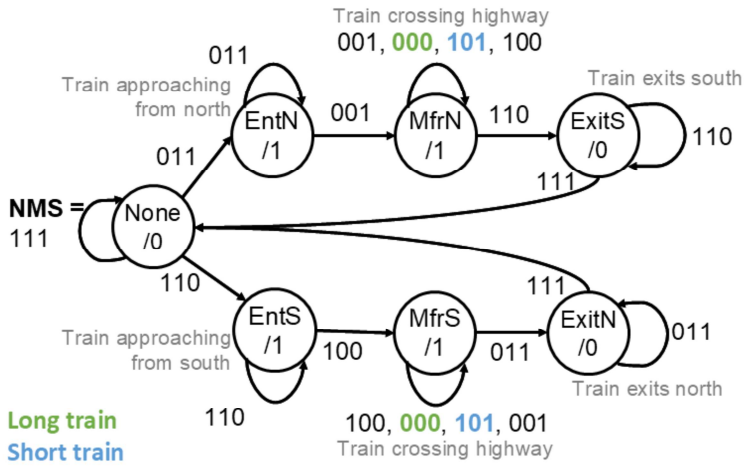
Next, let's consider the state where the train is crossing the highway. This includes the cases where **NMS = 100, 000, 101,** and **001**. From the **EntS** state, the system transitions to the **MfrS** state when **NMS = 100** and **F = 1,** indicating that the train is crossing the highway.

Drawing the state diagram



Now, let's consider the state where the train exits the North. We'll call this the **ExitN** state. From the **MfrS** state, the system transitions to **ExitN** when **NMS = 011** and **F = 0**, indicating that the train has exited the crossing but is still in the North section. Once the train is completely out (**NMS = 111**), the system will return to the **None** state.

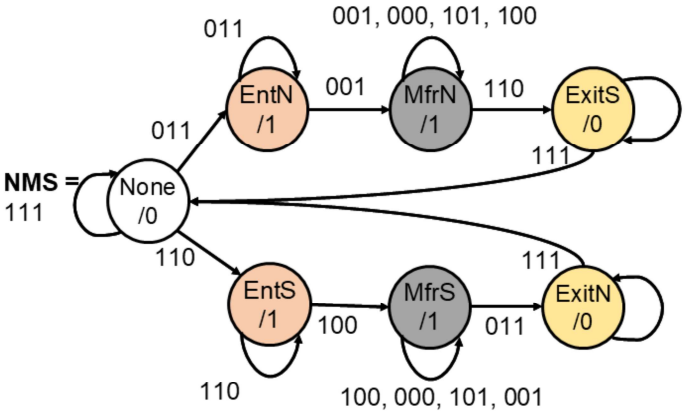
Drawing the state diagram



You may simplify the state diagram further or proceed to state minimization.

We've completed the state diagram, so now we can move on to state minimization or simplification of state diagram.

State minimization



Let's try to **simplify the state diagram further**.

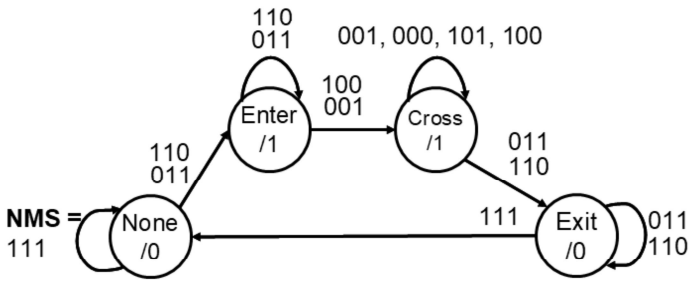
Notice that the two branches have similar behavior, only differing in direction.

Attempt to combine the 'entering' and 'exiting' states.

Notice that the two branches exhibit similar behavior, with the only difference being the direction of the train (entering from the North or South). Since both branches follow essentially the same pattern, we can attempt to combine the 'entering' and 'exiting' states into a single set of states that capture both scenarios.

By doing so, we simplify the state diagram and reduce redundancy. This means we no longer need separate states for trains entering from the North or South, as both will follow the same logic for crossing and exiting the highway. We can group these states together based on their shared transitions and behaviors, leading to a more efficient and compact representation of the system.

State minimization



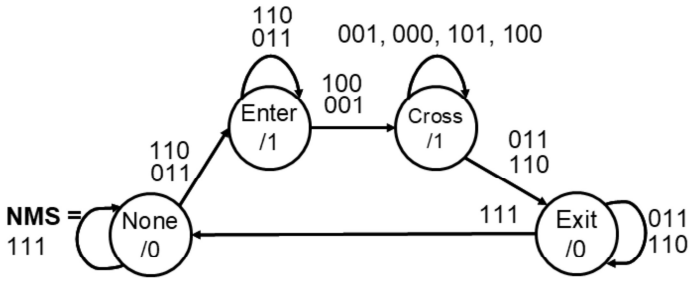
Let's try to **simplify the state diagram further**.

More thought into the state diagram reduced states from 7 to 4. (One FF less!)

Try doing the state minimization process using the techniques we learned in class as well!

Now that we have simplified the state diagram, careful analysis has allowed us to reduce the number of states from 7 to 4. This reduction not only makes the design more efficient but also eliminates the need for one flip-flop, which directly contributes to a more streamlined circuit.

Construct excitation table

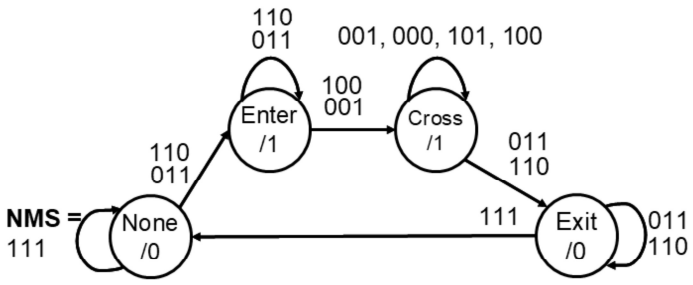


Using DFFs and assigning:
None = 00, Enter = 01, Cross = 11, Exit = 10:

PS	Next State for input NMS								F
Q_1Q_0	000	001	010	011	100	101	110	111	
00	xx	xx	xx	01	xx	xx	01	00	0
01	xx	11	xx	01	11	xx	01	xx	1
10	xx	xx	xx	10	xx	xx	10	xx	0
11	11	11	xx	10	11	11	10	xx	1

Recall: Don't cares used for situations that are not supposed to happen, e.g. 010 means train is in the north and south segments at the same time but not in the middle

Derive equations



Recall: Don't cares used for situations that are not supposed to happen, e.g. 010 means train is in the north and south segments at the same time but not in the middle

Using DFFs and assigning:
None = 00, Enter = 01, Cross = 11, Exit = 10:

PS	Next State for input NMS								F
Q_1Q_0	000	001	010	011	100	101	110	111	
00	xx	xx	xx	01	xx	xx	01	00	0
01	xx	11	xx	01	11	xx	01	xx	1
10	xx	xx	xx	10	xx	xx	10	xx	0
11	11	11	xx	10	11	11	10	xx	1

Using K-maps, we get the following expressions:

$$D_1 = N'Q_1 + M' + S'Q_1$$
$$D_0 = N'Q_1' + M' + S'Q_1'$$
$$F = Q_0$$

Let's move on to constructing the excitation table, which will help us determine the logic needed to drive the flip-flop input based on the current state and inputs. For reference, the states are as follows: **None = 00**, **Enter = 01**, **Cross = 11**, and **Exit = 10**.

Starting with the None state (00):

When **F = 0** (no train detected yet), the system starts in **NMS = 111**, where it stays in the **None** state (00). The next relevant inputs are **011** and **110**, indicating that a train is approaching from the North or South. In these cases, the system transitions to the **Enter** state (01). All other inputs are don't cares.

Moving to the Enter state (01):

Here, **F = 1**, as a train is detected. For inputs **110** and **011**, the system remains in the **Enter** state (01). However, when it detects **100** or **001**, the system moves to the **Cross** state (11). Other inputs are don't cares.

Now for the Cross state (11):

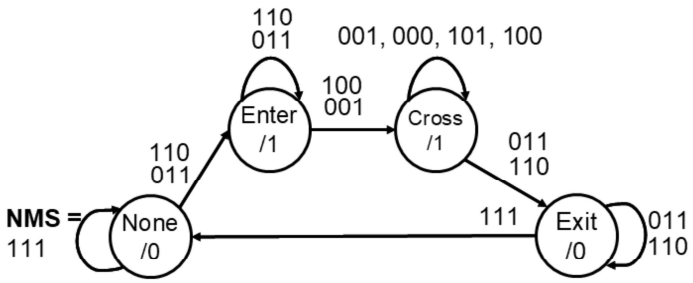
In this state, **F = 1**, as the train is still crossing. For inputs **001**, **000**, **101**, and **100**, the system stays in the **Cross** state (11). When it detects **011** or **110**, the

system transitions to the **Exit** state (10). Other inputs are don't cares.

Finally, the Exit state (10):

Here, **F = 0**, since the train has exited the crossing. For inputs **011** and **110**, the system remains in the **Exit** state (10). Once the input becomes **111**, the system transitions back to the **None** state (00).

Derive equations



Exercise:
Are the equations correct?
Does this work as intended?

Using DFFs and assigning:
None = 00, Enter = 01, Cross = 11, Exit = 10:

PS	Next State for input NMS								F
Q_1Q_0	000	001	010	011	100	101	110	111	
00	xx	xx	xx	01	xx	xx	01	00	0
01	xx	11	xx	01	11	xx	01	xx	1
10	xx	xx	xx	10	xx	xx	10	xx	0
11	11	11	xx	10	11	11	10	xx	1

Using K-maps, we get the following expressions:

$$D_1 = N'Q_1 + M' + S'Q_1$$
$$D_0 = N'Q_1' + M' + S'Q_1'$$
$$F = Q_0$$

Based on the excitation table we created earlier, we derived the excitation inputs and output equations, which are shown at the bottom left. As an exercise, verify if the equations are correct and ensure that the system functions as intended.